



线性表的查找

基本概念

顺序查找

折半查找

分块查找



线性表的查找

基本概念

顺序查找

折半查找

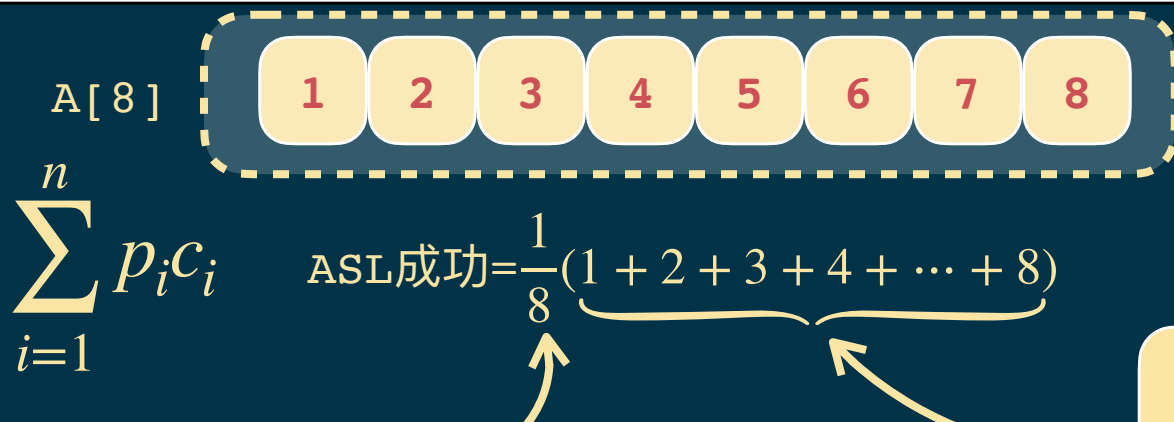
分块查找

查找表：由同一类型的数据元素(或记录)构成的集合

静态查找表：查找表的操作只有查找，不会被修改

动态查找表：需要支持插入或删除操作的查找表

平均查找长度ASL：为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值



假设等概率查找，每个位置查找概率相同，都是 $\frac{1}{8}$

第一个位置找到的话需要对比1次
第二个位置找到的话需要对比2次
...
第八个位置找到的话需要对比8次

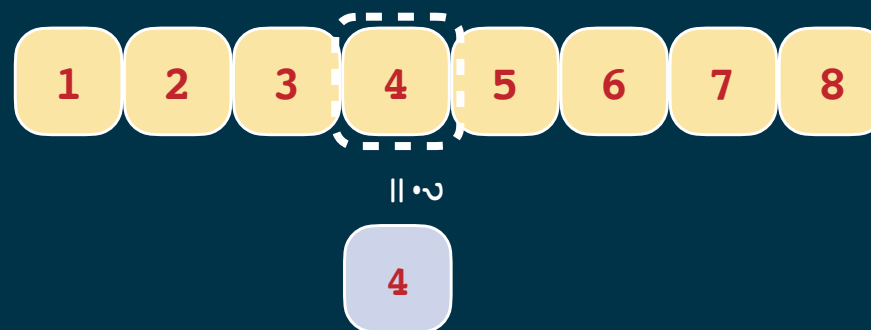


```
typedef struct{//数据元素类型定义
    KeyType key; //关键字域
    InfoType otherinfo; //其他域
}ElemType;
```

```
typedef struct{//顺序表的定义
    ElemType *R;//存储空间基地址
    int length; //当前长度
}SSTable;
```

```
int Search_Seq(SSTable ST, KeyType key){
// 在顺序表 ST 中顺序查找其关键字等于 key 的数据元素。
//若找到，则函数值为该元素在表中的位置，否则为0
    for (i=ST.length;i>=1;--i)
        if (ST.R[i].key==key) return i;
    return 0;
}
```

在该查找表中查找关键字4



查找成功，比较次数为4



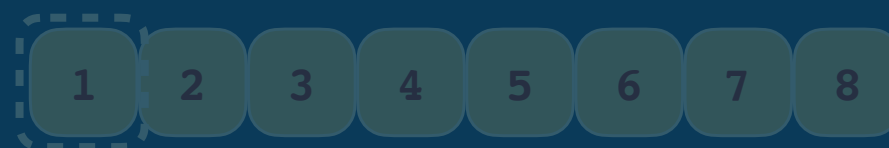
```
typedef struct{//数据元素类型定义
    KeyType key; //关键字域
    InfoType otherinfo; //其他域
}ElemType;
```

```
typedef struct{//顺序表的定义
    ElemType *R;//存储空间基地址
    int length; //当前长度
}SSTable;
```

```
int Search_Seq(SSTable ST, KeyType key)
// 在顺序表 ST 中顺序查找其关键字为key的元素
//若找到，则函数值为该元素在表中的位置，否则为0
{
    for (i=ST.length; i>=1; i--i)
        if (ST.R[i].key==key) return i;
    return 0;
}
```

每步都要有循环变量是否满足条件 $i \geq 1$ 的检测
比较麻烦，引入哨兵来去掉这一步条件

在该查找表中查找关键字9



// 哨兵

查找失败，比较次数为8

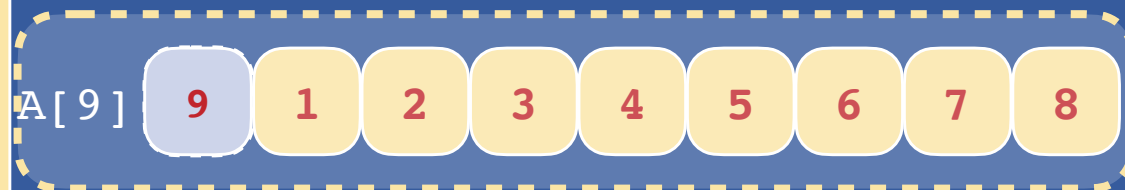


```
typedef struct{//数据元素类型定义
    KeyType key; //关键字域
    InfoType otherinfo; //其他域
}ElemType;
```

```
typedef struct{//顺序表的定义
    ElemType *R;//存储空间基地址
    int length; //当前长度
}SSTable;
```

```
int Search_Seq(SSTable ST, KeyType key) {
// 在顺序表 ST 中顺序查找其关键字等于 key 的数据元素
//若找到，则函数值为该元素在表中的位置，否则为0
    ST.R[0].key=key; //"监视哨"
    for(i=ST.length;ST.R[i].key!=key;--i); //从后往前找
    return i;
}
```

在该查找表中查找关键字9



将要查找的元素放在A[0]
这样返回的如果是0，就是查找失败
返回的是其他数字，那就是数组下标

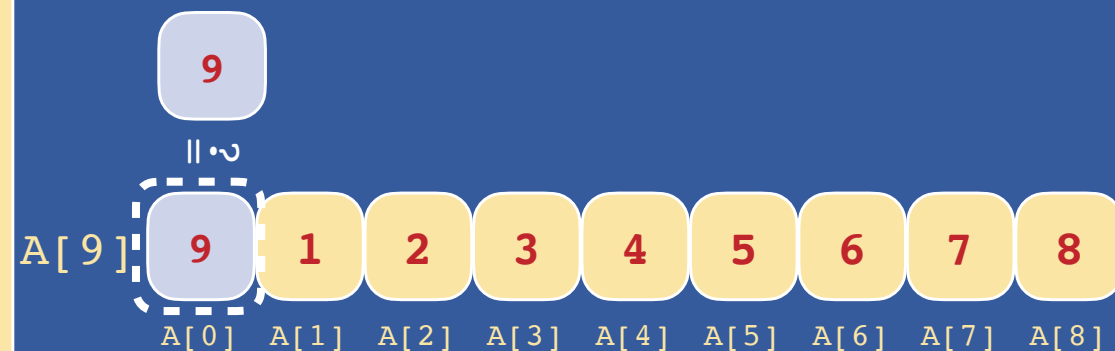


```
typedef struct{//数据元素类型定义
    KeyType key; //关键字域
    InfoType otherinfo; //其他域
}ElemType;
```

```
typedef struct{//顺序表的定义
    ElemType *R;//存储空间基地址
    int length; //当前长度
}SSTable;
```

```
int Search_Seq(SSTable ST, KeyType key) {
    // 在顺序表 ST 中顺序查找其关键字等于 key 的数据元素
    //若找到，则函数值为该元素在表中的位置，否则为0
    ST.R[0].key=key; //"监视哨"
    for(i=ST.length;ST.R[i].key!=key;--i); //从后往前找
    return i;
}
```

在该查找表中查找关键字9



查找失败，返回0，查找次数为9



线性表的查找

[基本概念](#)[顺序查找](#)[折半查找](#)[分块查找](#)

```
int Search_Seq(SSTable ST, KeyType key) {  
    // 在顺序表 ST 中顺序查找其关键字等于 key 的数据元素  
    // 若找到，则函数值为该元素在表中的位置，否则为0  
    ST.R[0].key=key; // "监视哨"  
    for(i=ST.length; ST.R[i].key!=key; --i); // 从后往前找  
    return i;  
}
```

时间复杂度： $O(n)$

优点：算法简单，对表结构无任何要求，适用于顺序和链式存储

缺点：平均查找长度较大，查找效率较低， 所以当n很大时，不宜采用顺序查找



线性表的查找

基本概念

顺序查找

折半查找

分块查找

使用要求：

1. 线性表必须用顺序存储
2. 线性表必须有序排列

```
int Search_Bin (STable ST, KeyType key) {  
    //在有序表 ST 中折半查找其关键字等于 key 的数据元素  
    low=1;high=ST.length;//置查找区间初值  
    while(low<=high){  
        mid=(low+high)/2;  
        if (key==ST.R[mid].key) return mid;  
        else if (key<ST.R[mid].key) high=mid-1;  
        else low=mid+1;  
    }  
    return 0;//表中不存在待查元素  
}
```



传入查找表，查找5

```
int Search_Bin (STable ST, KeyType key) {  
    //在有序表 ST 中折半查找其关键字等于 key 的数据元素  
    low=1;high=ST.length;//置查找区间初值  
    while(low<=high){  
        mid=(low+high)/2;  
        if (key==ST.R[mid].key) return mid;  
        else if (key<ST.R[mid].key) high=mid-1;  
        else low=mid+1;  
    }  
    return 0;//表中不存在待查元素  
}
```

查找关键字5

5

-	2	3	4	6	7	8
0	1	2	3	4	5	6



传入查找表，查找5

```
int Search_Bin (STable ST, KeyType key) {  
    //在有序表 ST 中折半查找其关键字等于 key 的数据元素  
    low=1;high=ST.length; //置查找区间初值  
    while(low<=high){  
        mid=(low+high)/2;  
        if (key==ST.R[mid].key) return mid;  
        else if (key<ST.R[mid].key) high=mid-1;  
        else low=mid+1;  
    }  
    return 0; //表中不存在待查元素  
}
```

返回0，查找失败

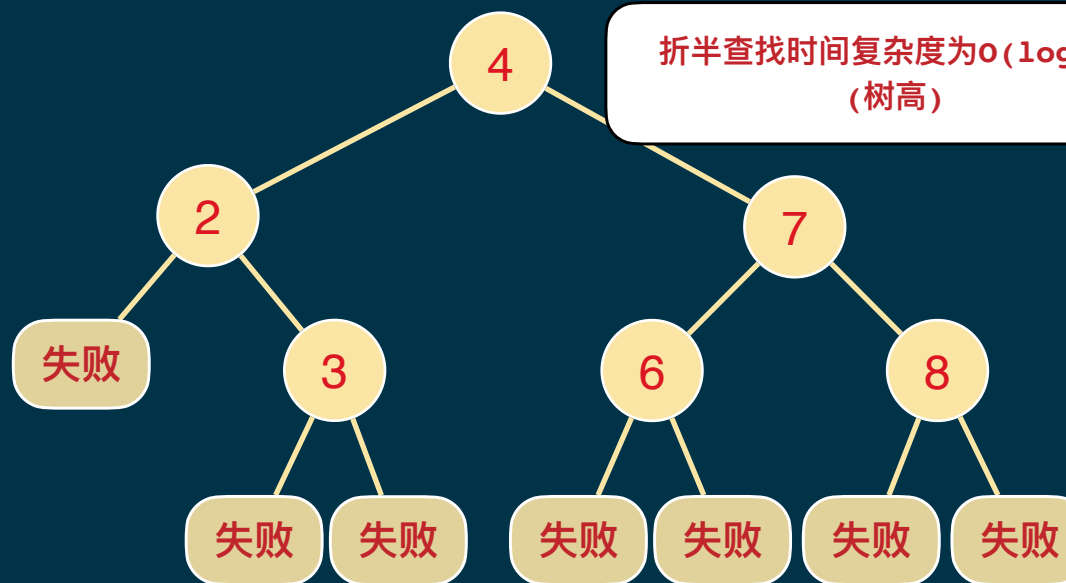
$$ASL_{成功} = (1+2+2+3+3+3) / 6 = \frac{14}{6} = \frac{7}{3}$$

$$ASL_{失败} = (2+3+3+3+3+3+3) / 7 = \frac{20}{7}$$

查找关键字5

-	2	3	4	6	7	8
0	1	2	3	4	5	6

对应查找树

折半查找时间复杂度为 $O(\log_2 n)$
(树高)

查找成功时的查找长度为从根节点到目的节点路径上的节点数
查找失败的查找长度为从根节点到对应失败节点的父节点路径上的节点数



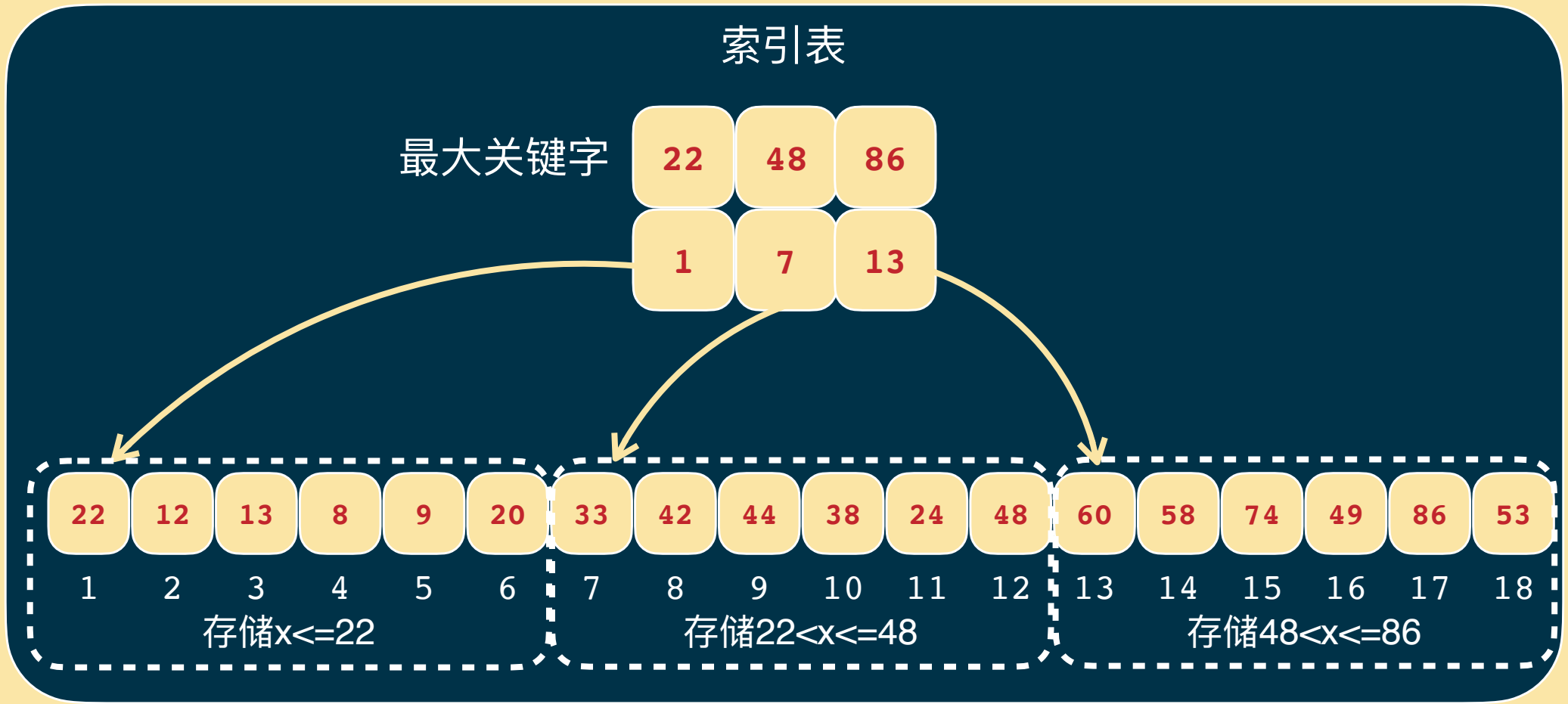
线性表的查找

基本概念

顺序查找

折半查找

分块查找
(索引顺序查找)





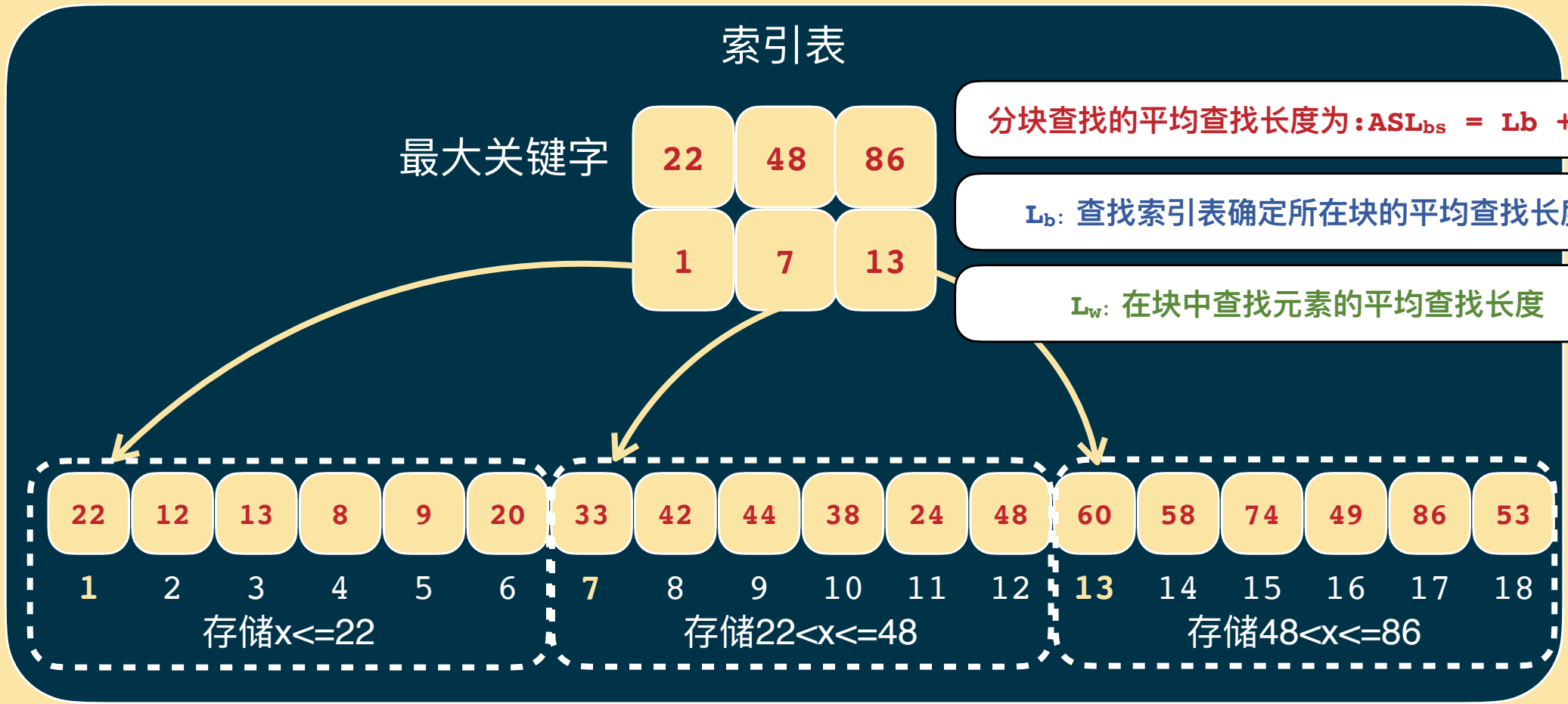
线性表的查找

基本概念

顺序查找

折半查找

分块查找
(索引顺序查找)



分块查找的平均查找长度为: $ASL_{bs} = L_b + L_w$

L_b : 查找索引表确定所在块的平均查找长度

L_w : 在块中查找元素的平均查找长度



索引表

最大关键字

22

48

86

1

7

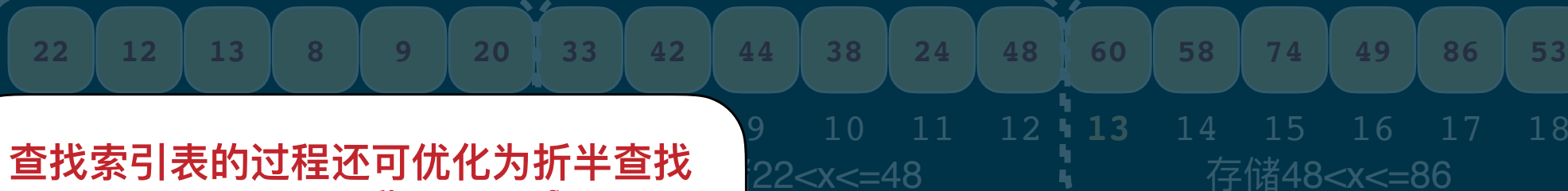
13

分块查找的平均查找长度为： $ASL_{bs} = L_b + L_w$

L_b : 查找索引表确定所在块的平均查找长度

L_w : 在块中查找元素的平均查找长度

每块查找的概率为 $1/b$ ，块中每个记录的查找概率为 $1/s$
长度为 n 的表均匀地分成 b 块，每块含有 s 个记录，即 $b = \lceil n/s \rceil$



查找索引表的过程还可优化为折半查找

$$ASL'_{bs} \approx \log_2 \left(\frac{n}{s} + 1 \right) + \frac{s}{2}$$

$$= \frac{1}{b} \sum_{j=1}^b j + \frac{1}{s} \sum_{i=1}^s i = \frac{b+1}{2} + \frac{s+1}{2} = \frac{1}{2} \left(\frac{n}{s} + s \right) + 1$$



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 【2010统考真题】已知一个长度为16的顺序表L，其元素按关键字有序排列，若采用折半查找法查找一个L中不存在的元素，则关键字的比较次数最多是（ ）。

A. 4

B. 5

C. 6

D. 7



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 【2015 统考真题】下列选项中，不能构成折半查找中关键字比较序列的是（）。

A. 500, 200, 450, 180

B. 500, 450, 200, 180

C. 180, 500, 200, 450

D. 180, 200, 500, 450